FROM FOUNDATIONS TO PRODUCTION

AI Engineering

An illustrated study guide

A practical explanation of every topic in your 13 roadmap screenshots, with examples, visual models and exercises.

01-03	Build the foundations
04-07	Create LLM applications and agents
08-10	Evaluate, observe and protect
11-13	Operate, deploy and design

Running example: a company knowledge and support assistant

Includes a capstone project, self-checks and linked primary references

PREPARED 12 OCTOBER 2026

YOUR LEARNING MAP

How to use this guide

Read chapters 1-3 for foundations, then build through chapters 4-7. Treat evaluation, security and operations as part of every project. Each concept includes an explanation and a practical example.

01	Programming & software engineering	3
02	AI / ML foundations	7
03	Generative AI & foundation models	11
04	LLM application engineering	14
05	RAG engineering	16
06	AI agents & MCP	20
07	Agent orchestration	23
08	AI evaluation engineering	26
09	AI observability	29
10	AI security & guardrails	31
11	GenAI Ops / AgentOps	34
12	Cloud AI engineering	36
13	AI system design	39
14	Capstone & practice	43
15	Sources & further reading	45

Reading notes. Examples and numbers are illustrative. Product names follow the screenshots, with naming changes noted where relevant. Explanations target a beginner-to-intermediate reader; code fragments show concepts rather than complete production systems.

Python: the working language

Python connects data processing, model calls and production services. Learn to make small pieces of code predictable before adding AI.

Fundamentals. Variables hold values; lists and dictionaries organize them; conditions, loops and functions express behavior. Prefer small functions with explicit inputs and outputs.
Example: A function cleans whitespace from each document and returns a list of non-empty paragraphs.

Object-oriented programming (OOP). Classes group state and behavior. Encapsulation keeps details behind an interface; composition combines smaller components. Inheritance is useful when a true shared contract exists.
Example: A KnowledgeAssistant contains a Retriever and a ModelClient, so either component can be replaced independently.

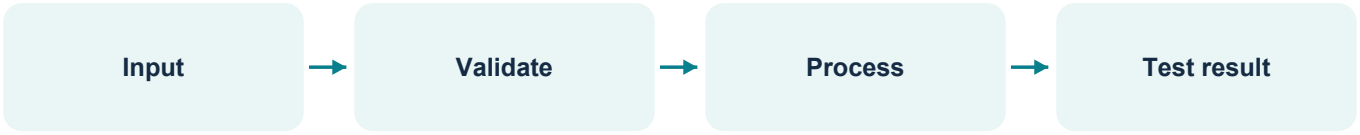
Async Python. async and await let a task yield while waiting for network or disk I/O. Concurrency helps overlap waits; it does not automatically accelerate CPU-heavy work.
Example: Fetch three independent policy documents concurrently, with a semaphore limiting requests to the document service.

Type hints. Annotations describe expected types to readers, editors and static checkers. Python usually does not enforce them at runtime; validation is a separate responsibility.
Example: def summarize(text: str) -> str tells a checker that the function expects and returns text.

Error handling. Raise meaningful exceptions, catch only failures you can handle, and release resources in finally blocks or context managers. Separate invalid input from temporary outages.
Example: A document parser reports an unsupported file type immediately but allows a timed-out download to be retried.

Testing. Unit tests check isolated behavior; integration tests check real boundaries; end-to-end tests check a complete flow. Use deterministic assertions around probabilistic model behavior.
Example: Test that an unauthorized document is never retrieved, and separately evaluate whether the generated answer is useful.

CONCEPT FLOW



In practice Practice: implement a typed document-cleaning function, then test empty input, Unicode text and malformed records.

Backend APIs: turn code into a service

An API is a contract between callers and your application. The server must validate inputs, enforce identity and make failures understandable.

REST APIs. REST commonly represents resources with HTTP methods and URLs. GET reads; POST creates or triggers work; status codes communicate outcomes. An API request should carry the context needed to process it.
Example: POST /tickets creates a ticket; GET /tickets/42 retrieves it; a missing ticket returns 404.

FastAPI. FastAPI builds Python HTTP APIs from route functions and typed request models, with generated OpenAPI documentation. Framework convenience does not replace domain validation or access control.
Example: Expose POST /ask, accept a question and return an answer object with citations. [1]

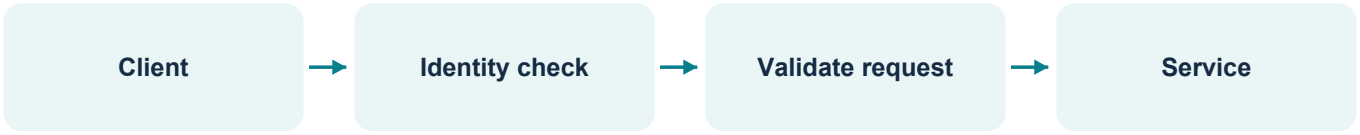
Pydantic. Pydantic parses and validates data against Python models. Constraints, enums and custom validators make contracts precise; strict validation can avoid unwanted type coercion.
Example: A Question model requires 1-2,000 characters and rejects unexpected fields rather than silently accepting them.

Authentication. Authentication establishes identity; authorization decides what that identity may do. Validate token signatures, issuer, audience and expiry where applicable, then check resource permissions.
Example: An employee can authenticate successfully but still cannot read another department's restricted documents.

WebSockets. A WebSocket keeps a bidirectional connection open for ongoing messages. It suits interactive sessions; simple one-way answer streaming may only need server-sent events.
Example: A voice assistant receives microphone events while sending transcription and response events over the same connection.

API design. Use consistent schemas, pagination, timeouts, versioning and stable error codes. Idempotency keys help prevent duplicate side effects when clients retry.
Example: Two retries with the same ticket-creation key return the original ticket instead of creating three tickets.

CONCEPT FLOW



In practice Design check: specify the success response, error response, timeout behavior and authorization rule before writing the endpoint.

[1] FastAPI documentation | Links in Sources & further reading.

Data: store facts, not just text

Data modeling determines whether information can be queried, updated and protected correctly.

SQL. Structured Query Language reads and changes relational data using operations such as SELECT, JOIN and GROUP BY. Parameterized queries keep values separate from query syntax.
Example: Count support tickets by status using GROUP BY status, while binding the current organization ID as a parameter.

PostgreSQL. PostgreSQL is a relational database with transactions, constraints and rich indexing. It is well suited to authoritative application records; extensions can add vector search.
Example: Store users, documents, permissions and tickets, with foreign keys preventing a ticket from referencing a nonexistent owner.

Redis. Redis provides low-latency data structures often used for caches, counters and short-lived state. Persistence and availability depend on deployment settings; do not assume it is automatically durable.
Example: Cache an expensive retrieval result for five minutes and maintain a per-user request counter.

JSON. JSON encodes objects, arrays, strings, numbers, booleans and null for interchange. It is a format, not a schema or a guarantee of correctness.
Example: An answer payload contains answer, citations and needs_review; the server checks these fields before returning it.

Data modeling. Choose entities, relationships, keys, constraints and access boundaries before implementation. Normalize shared facts where consistency matters and denormalize deliberately for measured read needs.
Example: A document has many chunks; every chunk keeps document_id, tenant_id, version and source location.

Entity	Important fields	Why it exists
Document	id, tenant_id, version, URI	Ownership and source of truth
Chunk	id, document_id, text, vector	Searchable evidence
Ticket	id, requester_id, status	Durable business action

In practice A vector match must still pass permission checks. Similarity is not authorization.

Development engineering and DevOps

DevOps combines development and operations practices so changes can be built, delivered and maintained reliably.

Git / GitHub. Git records versioned changes; branches isolate work and commits preserve history. GitHub adds hosted collaboration, pull requests and automation around repositories.
Example: A prompt change and its evaluation cases are reviewed together in one pull request.

Linux. Linux skills include navigating files, managing permissions, running processes, inspecting ports and reading service logs. These are everyday tools for diagnosing deployed systems.
Example: Use process and log inspection to discover that a worker is restarting because its memory limit is too small.

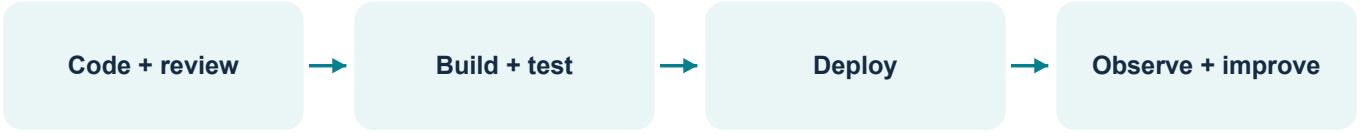
Docker. A container image packages application code and dependencies. Containers share a host kernel, so an image is not a complete virtual machine or a perfect security boundary.
Example: Build one API image, run it locally, then deploy the same immutable image digest to staging.

Environment management. Separate development, test and production settings. Pin dependencies, isolate Python environments and inject configuration at runtime rather than hard-coding it.
Example: Staging uses a test database and limited credentials while production uses independently managed secrets.

Logging. Structured logs record events with fields such as request_id, duration and error_type. Log enough to investigate failures while avoiding secrets and unnecessary personal data.
Example: A retrieval failure records its index version and trace ID, allowing the incident to be reproduced.

Cloud fundamentals. Understand compute, storage, networking, identity, regions, availability zones and shared responsibility. Managed services reduce some operations work without eliminating your responsibilities.
Example: Keep the application and database close in network distance and grant the API only the database permissions it needs.

CONCEPT FLOW



In practice Practice: containerize the API, configure it through environment variables and make one request traceable across its logs.

Machine learning: learn from examples

Machine learning fits patterns from data. Generalization to unseen cases matters more than performance on examples the model already saw.

Supervised ML. Learn a mapping from inputs to labeled outputs. Classification predicts categories; regression predicts numbers. Split data so the test set represents genuinely unseen cases.
Example: Train on labeled tickets to predict whether a new ticket concerns access, billing or hardware.

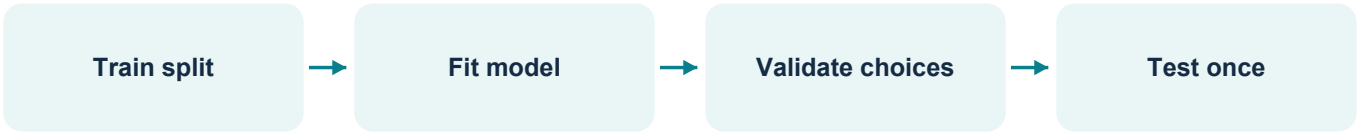
Unsupervised ML. Find structure without target labels, using methods such as clustering or dimensionality reduction. A discovered cluster still needs interpretation and validation.
Example: Group similar support requests to discover a recurring login problem that has no existing category.

Features. Features are the input representations a model uses. Preprocessing must be fitted on training data only, and features must be available at prediction time.
Example: Use ticket text and its age at arrival; do not use its eventual resolution time to predict urgency at arrival.

Evaluation. Choose metrics that reflect the cost of mistakes and test on held-out data. Inspect subgroups and distribution changes rather than relying on one average score.
Example: For urgent-ticket detection, inspect recall because missing an urgent case may matter more than reviewing an extra case.

Model serving. Serving makes predictions available through an endpoint, batch job or embedded runtime. Include preprocessing, model loading, monitoring and a versioned input/output contract.
Example: A deployed classifier returns both its category and model version so errors can be traced to a release.

CONCEPT FLOW



In practice Leakage example: randomly splitting nearly identical tickets from the same incident can inflate scores. Split by incident, user or time when the real deployment requires it.

Deep learning: representations and gradients

Neural networks learn representations through layers of numerical operations. Training changes parameters; inference uses the trained parameters.

Neural networks. Layers combine weighted inputs with nonlinear transformations. A loss function scores prediction error; parameters are adjusted to reduce that loss on training examples.
Example: A network converts a ticket embedding into probabilities for several support categories.

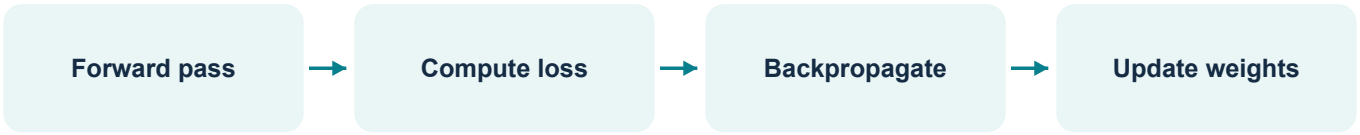
Backpropagation. Backpropagation applies the chain rule to compute how each parameter affects the loss. It produces gradients; an optimizer decides how to use them.
Example: If a correct category receives too little probability, gradients indicate parameter changes that can reduce that example's loss.

Optimizers. Methods such as SGD and Adam update parameters using gradients. Learning rate controls step size; batch size and regularization affect optimization and generalization.
Example: A learning rate that is too large makes validation loss unstable, so reduce it and compare against the same baseline.

PyTorch. PyTorch supplies tensors, automatic differentiation, neural-network modules and device execution. A training loop clears gradients, computes loss, backpropagates and updates weights.
Example: Use loss.backward() to compute gradients and optimizer.step() to update a small classifier. [2]

GPU basics. GPUs accelerate highly parallel tensor operations. Memory must hold weights, activations and other state; data transfer and poor batching can erase speed gains.
Example: If a training batch exhausts GPU memory, reduce its size or use gradient accumulation and appropriate mixed precision.

CONCEPT FLOW



In practice Training loop: zero_grad -> prediction -> loss -> backward -> step. Validation uses held-out examples and avoids weight updates.

[2] PyTorch: Learn the Basics | Links in Sources & further reading.

NLP and the transformer

Natural language processing (NLP) turns language into representations a computer can analyze and generate.

NLP basics. Common tasks include classification, named-entity recognition, translation, summarization and question answering. Language is ambiguous, so context and task-specific evaluation matter.
Example: Extract product names from a complaint, then classify the complaint's intent.

Tokenization. A tokenizer divides text into model-specific pieces such as subwords or bytes and maps them to IDs. Tokens are not reliably equivalent to words.
Example: A rare technical identifier may split into several tokens, affecting context usage and cost.

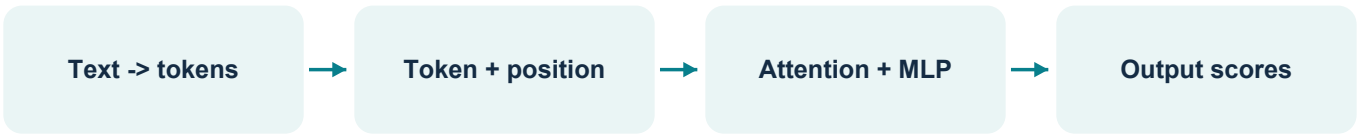
Embeddings. An embedding is a learned vector representation. Nearby vectors often capture useful similarity, but similarity depends on the model and is not proof of factual equivalence.
Example: The questions 'reset my password' and 'change a forgotten login secret' may retrieve the same help article.

Attention / self-attention. Attention combines value vectors using weights derived from queries and keys. Self-attention relates positions within the same sequence. A causal mask prevents a decoder from seeing future tokens.
Example: In 'the laptop overheated because it was blocked,' attention can help relate 'it' to surrounding context. [3]

Multi-head attention. Multiple attention heads use separate learned projections to capture different relationships, then combine their outputs. Heads do not necessarily map neatly to human-defined grammar rules.
Example: Different heads can represent local word relationships and longer-range references in the same sentence.

Positional encoding. Position information supplies order that plain attention lacks. Designs include learned embeddings and rotary representations, each affecting how position enters the network.
Example: 'Dog bites person' must be distinguishable from 'Person bites dog' even though the word set is the same.

CONCEPT FLOW



In practice Scaled dot-product attention: Attention(Q,K,V) = softmax(QK^T / sqrt(d_k))V. The softmax weights determine how much each value contributes.

[3] Attention Is All You Need | Links in Sources & further reading.

Transformer lifecycle: learn, adapt, use

Transformer architectures and training stages solve different problems. Understanding the distinction helps avoid unnecessary retraining.

Encoder / decoder. An encoder builds representations using input context; a causal decoder generates subsequent tokens. Encoder-decoder models transform one sequence into another. Many generative LLMs use decoder-only designs.
Example: Use an encoder for text embeddings, a decoder for answer generation, or an encoder-decoder for translation.

Pretraining. Pretraining learns broad representations from large datasets using a general objective, often predicting missing or next tokens. It is computationally intensive and does not guarantee current factual knowledge.
Example: A language model learns patterns of explanations before it ever sees your company's support handbook.

Fine-tuning. Fine-tuning adjusts a pretrained model using task-specific examples. It can improve style or task behavior but requires careful datasets and held-out evaluation.
Example: Adapt a classifier to your organization's ticket labels and verify that improvements hold on future tickets.

Inference. Inference executes the trained model to produce predictions. In autoregressive generation, the model repeatedly selects a next token based on prior context.
Example: The assistant generates a response one token at a time while retaining attention state in a KV cache.

Stage	Changes weights?	Typical input	Outcome
Pretraining	Yes	Large general corpus	Base capabilities
Fine-tuning	Yes	Task-specific examples	Adapted behavior
Inference	Usually no	Prompt / features	Prediction or answer
RAG at inference	No	Prompt + retrieved evidence	Contextual answer

In practice Practice: explain why updating a policy document can fix a RAG answer immediately, while a fine-tuned model requires a new training-and-release cycle.

Foundation models and model engineering

A foundation model can support many downstream tasks. Select and configure it around measured task requirements, not brand familiarity.

LLMs: GPT / Gemini / Claude. These are model families from OpenAI, Google and Anthropic. Capabilities, modalities, tools, availability and commercial terms vary by model and deployment.
Example: Compare candidate models on the same 100 support questions rather than assuming a provider is best for every task.

Open models. Open-weight models make weights available under specific licenses; that does not always mean fully open-source training data or unrestricted use. Hosting adds operational responsibility.
Example: A team hosts a permitted model privately when deployment control matters, then measures hardware and maintenance costs.

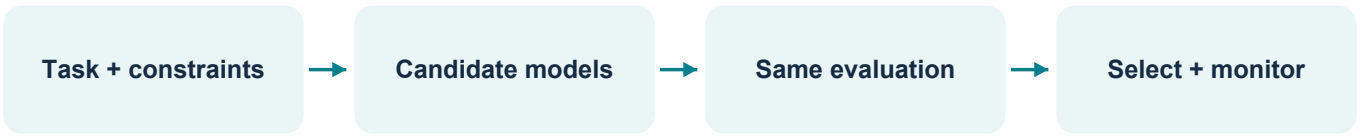
Tokens and context window. Tokens are processing units; the context window bounds what a request can accommodate under model-specific rules. Instructions, history, evidence and outputs consume budget.
Example: Reserve output capacity before packing retrieved documents, and truncate low-value history instead of dropping critical instructions.

Inference controls. Generation settings such as output limits and sampling affect behavior. Lower randomness can reduce variation but does not ensure correctness or full reproducibility.
Example: A classification service constrains output to a schema and a short response instead of requesting a long explanation.

API usage. A model API accepts a request and returns content or events. Handle authentication, timeouts, quotas and model-specific capabilities; keep credentials on the server.
Example: A backend adds a request ID, enforces an output limit and records usage without exposing its model credential to the browser.

Model selection. Measure quality, latency, cost, modality support, data requirements and operational fit on representative tasks. A smaller model can be sufficient for narrow, well-defined work.
Example: Use a compact model for routing tickets and a stronger model only for complex, evidence-heavy answers.

CONCEPT FLOW



In practice Product names follow the screenshots. This guide avoids fixed model rankings, context limits and prices because they change.

Adapt models and work across modalities

Model optimization changes resource needs or behavior. Multimodal applications also need careful parsing, alignment and validation.

Quantization. Quantization represents weights or activations with fewer bits to reduce memory and sometimes improve speed. Quality, hardware support and kernel efficiency determine the actual benefit.
Example: A 7-billion-parameter model needs roughly 14 GB for FP16 weights versus 3.5 GB at 4 bits, before overhead and runtime state.

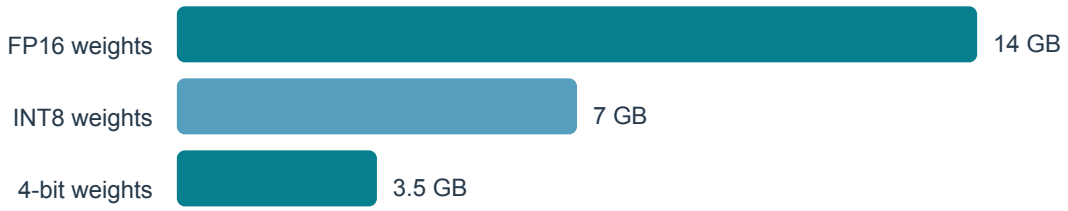
Fine-tuning for applications. Use fine-tuning when repeated examples can teach a desired behavior that prompting alone does not reliably achieve. Evaluate against the original model to detect regressions.
Example: Train on approved support replies to improve tone and classification consistency, while retrieving changing policies separately.

LoRA / PEFT. Parameter-efficient fine-tuning (PEFT) adapts a small parameter subset. LoRA learns low-rank update matrices while freezing base weights, reducing trainable parameter requirements.
Example: Keep one base model and separate lightweight adapters for different ticket taxonomies. [4]

Vision. Vision models interpret images or image-text combinations. Resolution, cropping and visual ambiguity influence accuracy; preserve the relevant source region.
Example: Read an error dialog from a screenshot and verify its exact error code before searching for a fix.

Audio and speech. Audio includes non-speech sounds; speech tasks include transcription, speech understanding and text-to-speech. Noise, accents, timing and speaker identity affect the pipeline.
Example: Transcribe a spoken support request, ask the user to confirm a critical serial number, then speak the answer.

Documents. Documents combine text, images, layout, tables and reading order. OCR alone may lose structure; retain page and coordinate information where useful.
Example: Extract a pricing table with its column headers and page reference rather than merging all numbers into a flat sentence.



Illustrative 7B model: weights only; excludes scales, KV cache, activations and runtime.

[4] LoRA paper | Links in Sources & further reading.

Context engineering: choose what the model sees

Prompt engineering writes instructions. Context engineering assembles the instructions, examples, evidence, memory and tool results for each request.

Prompt engineering. State the task, constraints, desired format and success criteria clearly. Separate instructions from quoted data and evaluate changes against real cases.
Example: 'Answer using the supplied policy excerpts; cite their IDs; state when evidence is insufficient.'

System prompts. System-level instructions define application behavior, role and boundaries according to the provider's instruction hierarchy. They guide behavior but are not a substitute for server-side controls.
Example: The assistant is told to draft ticket changes; the backend still enforces permissions before a change can execute.

Few-shot prompting. Provide a few representative input-output examples to demonstrate a pattern. Include edge cases without using the evaluation answers as demonstrations.
Example: Show one answer with citations and one justified 'not enough information' response.

Structured context. Use explicit sections and identifiers so the model can distinguish task, data, examples and tool results. Structured context need not be JSON, but it should be unambiguous.
Example: Each excerpt includes source_id, version, title and text, inside a clearly marked evidence section.

Dynamic context. Assemble relevant information at request time based on identity, task and current state. Static prompts alone cannot represent changing facts.
Example: Load today's applicable travel policy for the authenticated employee's region.

Context compression. Summarize or shorten content to fit a budget, preserving constraints, provenance and unresolved issues. Compression can discard crucial detail, so test its effect.
Example: Replace an old conversation with a summary that retains the ticket ID, agreed action and pending approval.

Context selection. Rank information by relevance, authority, freshness and permission, then deduplicate. More context is not automatically better.
Example: Include the current policy and its exception clause; omit five duplicate copies of the same paragraph.

Long-context strategies. Use budgets, clear structure, staged retrieval and targeted summaries. Large windows still have cost, latency and attention-quality trade-offs.
Example: Search a long manual for candidate sections, then compare only those sections and retain their original references.

In practice Example budget (illustrative): 12,000 tokens = 1,000 instructions + 2,000 history + 6,000 evidence + 1,000 tool data + 2,000 reserved output. Verify actual accounting for the chosen model.

Reliable LLM interfaces

Treat model output as data crossing a trust boundary. The application remains responsible for validation and execution.

Structured outputs. Structured output constrains a response to a machine-readable contract, often using a supported schema. Structural validity does not prove that field values are correct.
Example: An answer object has text, source_ids and needs_review; a separate check verifies that cited sources actually support the answer.

JSON schemas. A JSON Schema describes allowed types, properties, required fields and constraints. Provider support can cover only a subset, so validate again in your own application.
Example: Require priority to be one of low, normal or high, and reject unknown properties in a ticket draft.

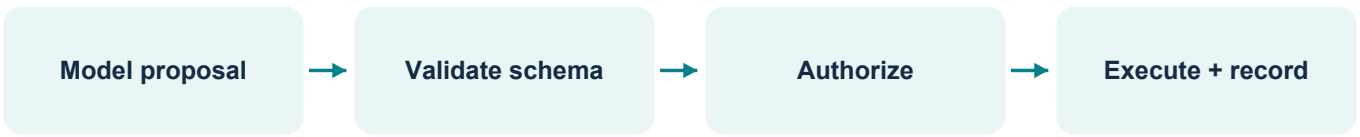
Function / tool calling. A model proposes a named tool and arguments. The host validates the request, checks authority, runs the tool and returns its result to the model.
Example: The model proposes get_ticket with ticket_id 42; the server first verifies that the caller can read ticket 42.

Streaming. Streaming delivers partial output or events before completion, improving perceived responsiveness. Partial JSON and incomplete tool arguments are not safe to execute.
Example: Show draft answer text progressively, but enable 'Create ticket' only after the complete validated result is available.

Async execution. Overlap independent I/O with bounded concurrency and cancellation. Respect dependencies and deadlines; one slow branch should not consume resources indefinitely.
Example: Fetch document metadata and user preferences together, but wait for authorization before retrieving restricted document text.

```
# Illustrative contract; not a complete endpoint
{
  "answer": "Follow the password reset procedure.",
  "source_ids": ["help-17-v3"],
  "needs_review": false
}
```

CONCEPT FLOW



In practice Never execute a tool merely because the model produced well-formed arguments. Check ownership, scope and side-effect policy in code.

Resilience, routing and cost control

Production traffic brings timeouts, changing demand and repeated questions. Control these at the application boundary.

Retry / fallback. Retry transient failures with capped exponential backoff, jitter and a deadline. Fallbacks need compatible quality and data policies. Avoid retrying invalid requests or duplicating side effects.
Example: Retry a rate-limited read after the permitted delay; retry ticket creation only with an idempotency key.

Model routing. Choose a model according to task requirements, measured complexity and operating constraints. Routing itself must be evaluated for mistakes and overhead.
Example: Send simple category labeling to a small model and policy-conflict analysis to a stronger model.

Semantic caching. Reuse a prior answer when a new query has sufficiently similar meaning and compatible context. Include tenant, permissions, source version and policy in cache eligibility.
Example: Reuse a public FAQ answer for paraphrases, but never reuse one employee's personalized payroll response for another.

Rate limiting. Limit requests, tokens, concurrency or spending over time. Client throttling protects the service; server quotas protect tenants and budgets. Return clear retry guidance.
Example: Allow a user a bounded burst while a separate organization-wide token budget prevents runaway loops.

LLM gateways. A gateway centralizes model access, credentials, quotas, logging and provider routing. It becomes a critical component whose availability and compatibility need attention.
Example: Applications call one internal endpoint while the gateway applies tenant budgets and routes to approved model deployments.

Cost optimization. Reduce unnecessary calls, context and output; cache safely; batch offline work where appropriate. Compare cost per successful outcome, not only cost per call.
Example: A cheaper model that requires three corrections may cost more overall than one accurate response from a stronger model.

Failure	Useful response	Avoid
Temporary timeout	Bounded retry + deadline	Infinite retry loops
Schema violation	Validate; repair or reject	Trusting malformed data
Provider outage	Tested fallback or clear failure	Silent quality downgrade
Duplicate write	Idempotency key + stored result	Repeating the side effect

In practice Cost = input_tokens x input_rate + output_tokens x output_rate + tool, retrieval and hosting costs. Match units, including any per-million-token pricing.

RAG ingestion: make evidence searchable

Retrieval-augmented generation (RAG) supplies external evidence at answer time. Its quality starts with the ingestion pipeline, before a model sees a question.

Parsing. Parsing extracts logical content from source formats such as HTML, PDF or Word. Preserve headings, tables and reading order; record parser errors and source identity.
Example: Convert a handbook into sections while retaining the hierarchy 'Travel > Meals > Exceptions'.

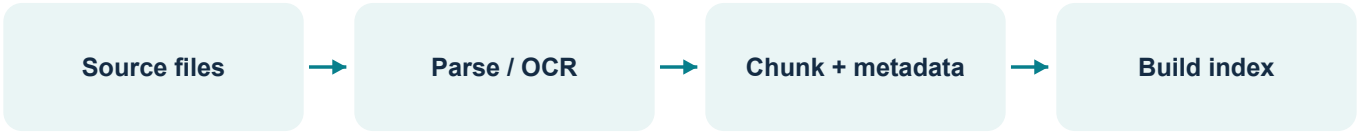
OCR. Optical character recognition extracts text from scanned images. Quality depends on resolution, layout and image quality, so critical numbers may require verification.
Example: A scanned reimbursement limit is recognized as 800; compare the source image before accepting a low-confidence extraction.

Chunking. Divide documents into retrievable units that retain enough meaning. Choose size and overlap experimentally; respect headings, table boundaries and semantic units.
Example: Keep one complete password-reset procedure together rather than separating its warning from its action steps.

Metadata. Metadata describes a chunk’s source, ownership, permissions, dates, language and version. It enables filtering, citation and lifecycle management.
Example: Store tenant_id, document_id, page, section, effective_date and access group alongside every chunk.

Indexing. An index organizes content for efficient lookup, using lexical terms, vectors or both. Updates and deletions must propagate so stale or revoked content disappears.
Example: When policy version 4 replaces version 3, index the new content and retire old chunks using a controlled version switch.

CONCEPT FLOW



In practice Worked example: one 20-page handbook becomes 80 section-aware chunks. Each chunk points back to its page and document version. These counts are illustrative, not a recommended chunking formula.

[5] Original RAG paper (conceptual background) | Links in Sources & further reading.

RAG retrieval and grounded generation

Retrieval finds candidate evidence; generation uses it to answer. Evaluate these stages separately so you know which stage failed.

Embeddings for retrieval. Encode queries and passages into compatible vectors using an embedding model. Changing models usually requires re-embedding indexed content.
Example: Embed 'How do I reset my password?' and compare it with help-article vectors from the same compatible embedding space.

Vector search / similarity search. Vector search ranks items by a distance or similarity measure, such as cosine similarity. Similarity search is the broader task; approximate methods trade some recall for speed.
Example: Retrieve the closest policy passages, then inspect whether they actually answer the question rather than merely share vocabulary.

Vector DB. A vector database stores vectors and supports indexing, filtering and updates. Some relational databases offer these functions through extensions, reducing infrastructure count.
Example: Store embeddings beside document metadata and query a vector index restricted to the caller's organization.

Metadata filters. Filters restrict candidates using attributes such as tenant, language, effective date or access scope. Enforce permissions before content reaches the model.
Example: Search only current English policies that the authenticated user is allowed to read.

Context building. Select, deduplicate and order retrieved passages within a token budget. Include identifiers and enough surrounding text to interpret each passage.
Example: Send the policy clause and its exception paragraph together, clearly labeled as sources A and B.

Citations and grounding. Citations point to evidence; grounding means claims are supported by that evidence. A plausible citation can still be irrelevant or wrong.
Example: The answer cites page 7 for a reimbursement limit; a check confirms page 7 contains the claimed limit and applies to this employee.

Answer generation. The model synthesizes evidence into an answer, handling conflicts and missing information explicitly. Retrieval does not eliminate hallucination.
Example: If the documents do not state an exception, answer that it is not established by the available sources and suggest escalation.

CONCEPT FLOW



In practice Cosine similarity = (q dot d) / (||q|| x ||d||). For q=(1,0) and d=(0.8,0.6), similarity is 0.8; that number is not a calibrated probability of relevance.

Advanced RAG: retrieve better candidates

Add retrieval techniques to solve observed failures. More stages increase latency, complexity and evaluation needs.

Hybrid search. Combine lexical retrieval, which matches terms, with vector retrieval, which matches learned meaning. Fuse rankings rather than assuming raw scores are comparable.
Example: Lexical search finds exact error code E1042 while vector search finds articles describing the same symptom differently.

Reranking. A second model scores a small candidate set more carefully, often considering each query-passage pair jointly. It improves ordering but cannot recover a missing candidate.
Example: Retrieve 40 passages cheaply, rerank them and include the top five that meet the evidence criteria.

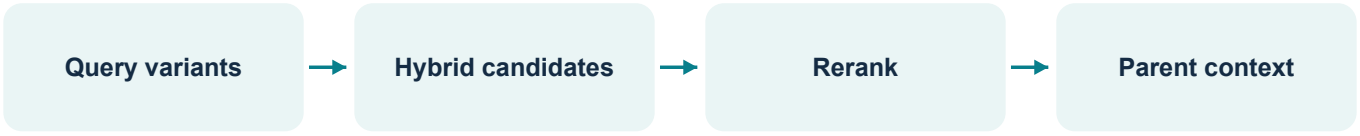
Query expansion. Add useful synonyms, aliases or related terms without changing the original intent. Too much expansion can reduce precision.
Example: Expand 'SSO login' to include 'single sign-on' and the organization’s identity-provider name.

Query rewriting. Reformulate a query into a clearer or standalone form using conversation context. Preserve the user’s meaning and important constraints.
Example: Rewrite 'Does that apply to contractors?' as 'Does the remote-work equipment policy apply to contractors?'

Multi-query RAG. Generate several complementary queries, retrieve for each and merge deduplicated results. This broadens recall at the cost of extra retrieval.
Example: Search separately for a policy’s eligibility rule, exception process and approval authority.

Parent-child retrieval. Index small child chunks for precise matching, then return a larger parent section for context. Preserve the relationship during updates.
Example: Match one sentence about a deductible, but supply the entire insurance section containing its conditions.

CONCEPT FLOW



In practice Debugging rule: low recall suggests ingestion or candidate-search issues; good recall with poor answers suggests context selection, grounding or generation issues.

Advanced RAG: richer evidence and control

These terms describe families of techniques. Research-specific methods are not interchangeable with a generic prompt that says to reflect.

Graph RAG. Represent entities and relationships as a graph and retrieve linked evidence or graph summaries. It can help relationship-heavy questions but adds extraction and maintenance costs.
Example: Connect service A to its owner, dependencies and incidents to answer which teams are affected by a failure.

Multimodal RAG. Retrieve evidence across text, images, tables, audio or video and preserve alignment with original sources. Retrieval and generation must support the required modalities.
Example: Find a machine diagram and its maintenance instructions, then explain the labeled valve using both sources.

Corrective RAG. Corrective approaches assess retrieved evidence and decide whether to refine retrieval, use another source or abstain. The CRAG paper proposes a particular retrieval-evaluation framework.
Example: Weak internal evidence triggers another approved source search instead of immediately generating a confident answer. [6]

Self-RAG. The Self-RAG research method trains a model to use reflection tokens for retrieval and critique. Broader self-checking pipelines exist, but a critique prompt alone is not that trained method.
Example: A trained system decides whether retrieval is needed, then assesses support for its proposed response. [7]

Agentic RAG. An agent chooses retrieval tools or follow-up queries dynamically. Bound its iterations and require evidence-based stopping criteria to control cost and drift.
Example: The assistant searches the handbook, detects a missing regional exception, searches the regional policy and then answers.

Observed problem	Candidate technique	Trade-off
Exact IDs missed	Hybrid search	Two retrieval signals
Relevant passage ranked low	Reranking	Extra model latency
Relationships span documents	Graph RAG	Graph maintenance
Evidence is weak	Corrective retrieval	More calls or abstention
Question needs several searches	Agentic RAG	Variable execution path

In practice Practice: record 20 failed questions, label the failure stage, and add one technique only if it improves those cases without regressing the baseline.

[6] Corrective RAG; [7] Self-RAG | Links in Sources & further reading.

AI agents: tools connect models to actions

An agent uses a model to choose actions toward a goal, observes tool results and decides what to do next. The host controls the available tools and execution boundaries.

APIs as tools. Wrap business services in narrow, documented functions with validated arguments and predictable results. Return meaningful errors the agent can handle.
Example: get_order_status(order_id) returns a structured status instead of exposing an unrestricted HTTP client.

Database tools. Prefer scoped queries or approved operations over broad arbitrary database access. Parameterize inputs and enforce row-level ownership.
Example: The assistant can list the caller's open tickets, but cannot run arbitrary SQL against the whole customer database.

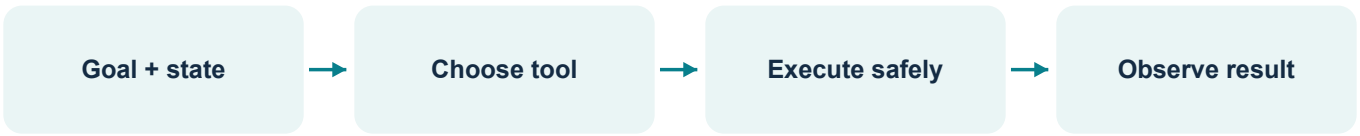
Search tools. Search retrieves candidates from approved corpora or the web. Results are evidence to evaluate, not instructions the agent should automatically obey.
Example: The agent searches official product documentation and checks the retrieved page's date and applicability.

Browser tools. Browser automation reads and interacts with web interfaces when APIs are unavailable. UI state can change, so verify the target and the result of each significant action.
Example: The assistant fills a ticket form and presents a draft before committing an action that requires approval.

Code tools. A sandboxed code runtime can perform calculations, data transformations or tests. Limit filesystem, network and resource access according to the task.
Example: Compute ticket-resolution statistics from an uploaded CSV inside an isolated runtime with no production credentials.

File tools. File tools read and write permitted paths with size and type limits. Keep provenance and protect against unintended overwrites or path traversal.
Example: Read a user-provided handbook and save a generated summary only in the designated output folder.

CONCEPT FLOW



In practice A tool call is a request for execution, not proof that the action succeeded. Verify the returned state and record the outcome.

Agent memory and reasoning patterns

Memory stores useful information across steps or sessions. Reasoning patterns organize the work; neither guarantees correctness.

Short-term memory. Working context contains the current conversation, recent tool results and active task state. It is limited by context and persistence choices.
Example: Keep the ticket ID and the most recent API response available during a troubleshooting session.

Long-term memory. Persistent storage retains information beyond a single run. Retrieval rules, retention, correction and access controls determine whether it remains useful.
Example: Store approved preferences between sessions, with a way for the user to inspect, update or delete them.

Episodic memory. Episodic memory records particular past events or interactions, including what happened and its outcome.
Example: Recall that a previous attempt to reset this device failed with a specific error and avoid repeating the same unsuccessful step.

Semantic memory. Semantic memory stores facts or generalized knowledge, with provenance and validity when possible. It differs from a record of one event.
Example: Remember that a product's maintenance interval is documented as 90 days, linked to the source version.

User memory. User memory captures personal preferences or relevant user facts with appropriate consent and isolation. It may contain episodic or semantic information.
Example: Remember that a user prefers concise explanations, without treating that preference as permission for external actions.

Planning and task decomposition. Break a goal into smaller steps with dependencies, success conditions and a stopping rule. Update the plan when evidence changes.
Example: To resolve a login issue: identify the account, check service status, retrieve the procedure and propose the next step.

Reflection. Reflection reviews an intermediate result for omissions or errors. It works best with independent evidence, executable checks or explicit rubrics.
Example: After drafting an answer, verify each numeric claim against its cited source instead of merely asking whether it sounds correct.

ReAct. ReAct alternates reasoning-guided action with observations. Applications can log concise decisions, tool calls and results without exposing hidden internal reasoning.
Example: Search for a policy, inspect the result, then perform a second targeted search only if an essential detail is missing.

In practice Memory is an application design choice, not automatic model learning. Saving a conversation does not update model weights, and stale memory should not override current evidence.

MCP: a shared interface for external capabilities

The Model Context Protocol (MCP) standardizes how AI applications connect to external capabilities. It does not itself decide what the user has authorized. [8]

MCP client. A client is the protocol component within a host application that connects to an MCP server and exchanges messages. The host manages model interaction and user-facing policy.
Example: A desktop assistant creates a client connection to an approved document server.

MCP server. A server exposes capabilities such as tools, resources and prompts. It may run locally or remotely, with transport and authentication appropriate to the deployment.
Example: A company server offers document search and retrieval without exposing its internal database credentials.

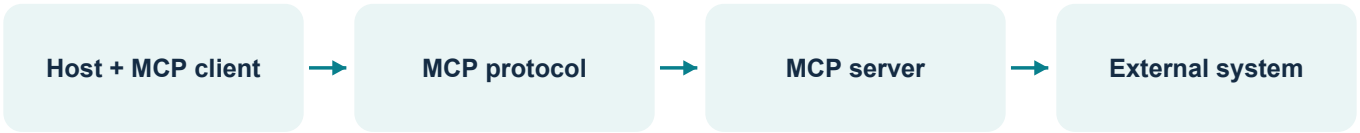
Tools. Tools are named callable operations with described inputs and results. Discovery makes them available; host policy determines when they may run.
Example: search_handbook(query) returns matching excerpts; create_ticket(payload) changes business state and needs stronger control.

Resources. Resources expose contextual content identified through the protocol, rather than inherently performing an action. The client chooses how to incorporate content.
Example: A resource URI identifies a policy document whose text can be included in the model's context.

Prompts. Prompts are reusable interaction templates supplied by a server. Review their provenance and treat them within the host's instruction and permission model.
Example: A troubleshooting template asks for the product, symptoms and recent changes before proposing diagnostic steps.

Access, authentication and safety. Authenticate identities, authorize each operation, scope tokens and validate destinations. A trusted connection does not make every returned document or tool description trustworthy.
Example: A read-only employee token can search permitted documents but cannot create admin accounts or expose another tenant's content.

CONCEPT FLOW



In practice Screenshot note: the far-right shield card is partly obscured. Its visible text concerns access, authentication and safety; this guide covers that theme without assuming its hidden heading.

[8] MCP architecture | Links in Sources & further reading.

Stateful orchestration and workflows

Orchestration coordinates steps, state and recovery. Use a predefined workflow when the steps are known; add model-driven choices where they create measurable value.

State. State is the explicit record of the run: goal, inputs, intermediate results, decisions and status. Prefer a typed structure over relying on the chat transcript alone.
Example: A run stores question, retrieved_ids, draft_answer, approval_status and final_ticket_id.

Persistence. Persistence saves state durably beyond the process lifetime. Plan schemas, retention, encryption and concurrent updates.
Example: A worker restart reloads the run from a database instead of forgetting that the user already approved a draft.

Checkpoints. A checkpoint captures a recoverable point in execution. Resuming still requires idempotent side effects, because a crash can occur after an action but before recording it.
Example: Save state after retrieval and after ticket creation; reconcile the idempotency key on recovery.

Context. Context is the subset of state and evidence sent to the model. Store raw state separately from its prompt representation so it can be rebuilt safely.
Example: A run keeps all retrieval metadata, but sends only five authorized passages to the answering model.

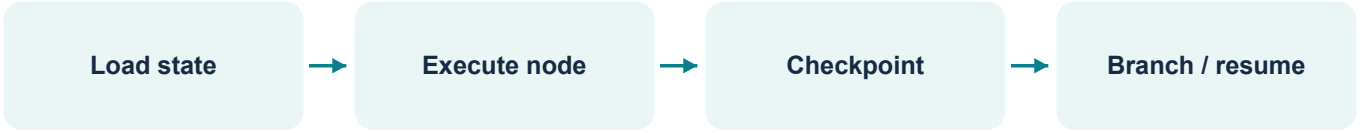
Sequential workflows. Execute steps in a fixed order when later steps depend on earlier results.
Example: Parse a document, then chunk it, then embed the chunks, then index them.

Parallel workflows. Run independent branches concurrently, with bounded fan-out and a defined merge rule. Shared writes may need conflict handling.
Example: Search two approved repositories at once and deduplicate the combined candidates.

Conditional workflows. Branch on explicit conditions or validated classification outputs. Make fallback behavior clear for ambiguous cases.
Example: If evidence is sufficient, answer; otherwise request more information or escalate.

Loops and event-driven workflows. Loops repeat until a condition or budget is reached. Event-driven workflows start or resume in response to an event, which may be delivered more than once.
Example: Retry document processing at most twice; resume approval processing when an approval event arrives, deduplicating repeated events.

CONCEPT FLOW



In practice A checkpoint does not undo a sent email, submitted payment or created ticket. Recovery must account for effects already committed outside the workflow.

Routing, supervision and collaboration

Multi-agent designs divide responsibilities. They can improve specialization, but extra handoffs also add coordination cost and new failure modes.

Agent selection. Choose a specialist according to task domain, authority and evaluated competence. Agent names alone do not establish capability.
Example: Route an access problem to the identity-support specialist and an expense-policy question to the policy specialist.

Tool selection. Choose the narrowest suitable operation based on required inputs and side effects. Distinguish reading information from changing state.
Example: Select get_ticket_status to answer a status question; do not use update_ticket unless a change is requested.

Dynamic routing / model routing. Dynamic routing adapts the next path to runtime state; model routing chooses the model for a step. Use explicit budgets and observed performance.
Example: Escalate a low-evidence answer to a deeper retrieval path while keeping simple extraction on a smaller model.

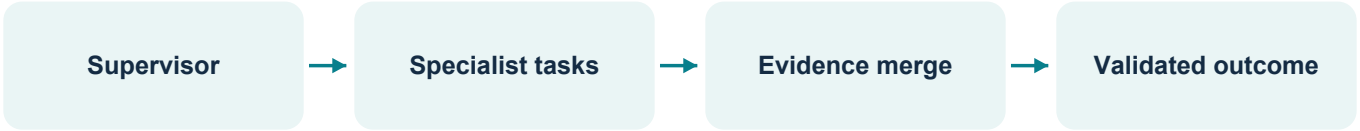
Supervisor agent. A supervisor decomposes work and delegates to workers or specialists, then checks and combines results. It should have a clear stop condition and conflict-resolution rule.
Example: A planner asks one worker for policy evidence and another for system status, then produces a single supported response.

Planner-worker / manager-specialists. Planner-worker separates task planning from execution; manager-specialists coordinates domain experts. Both require explicit input/output contracts.
Example: A manager requests an evidence bundle containing source IDs and supported claims, instead of accepting an unstructured 'done'.

Handoffs. A handoff transfers task ownership or control between agents, or from an agent to a human. Include task state, evidence, permissions and pending actions.
Example: The triage agent hands a case to a billing specialist with the user’s question, ticket ID and verified account scope.

Collaboration. Collaborating agents share a goal and exchange results or memory. Control concurrent writes, source provenance and disagreement; agreement is not proof of truth.
Example: Two specialists propose conflicting answers, so the supervisor checks the original policy rather than taking a majority vote.

CONCEPT FLOW



In practice Start with one agent or a workflow. Add multiple agents when independent expertise or parallel work improves measured outcomes enough to justify overhead.

Human control and framework choices

Human-in-the-loop design defines where people can approve, review, redirect or stop execution. Frameworks help implement this, but policy remains your responsibility.

Approval. Approval gates a specific proposed action before execution. Bind approval to its exact payload and scope; expire or renew it when material details change.
Example: Show the ticket recipient, content and attachments before an authorized person approves submission.

Review. Review checks an artifact or result against a rubric. It can occur before or after low-risk actions, depending on the process.
Example: A support lead reviews sampled answers for correctness and source quality.

Escalation and intervention. Escalation transfers an unresolved or sensitive case to an appropriate person. Intervention lets a person pause, change or cancel an active run.
Example: A repeated tool failure opens a review case, and the operator stops further retries while preserving the trace.

LangGraph. LangGraph models stateful execution through graph nodes and edges, with facilities for persistence, durable execution and human intervention.
Example: Use a graph with retrieve, draft and approve nodes so a stopped run can resume from saved state. [9]

OpenAI Agents SDK. The Agents SDK provides building blocks for agents, tools, handoffs, guardrails and tracing. Evaluate how its abstractions fit your execution and deployment needs.
Example: Build a triage agent that hands a supported case to a specialist and records tool activity for debugging. [10]

Google ADK. Google's Agent Development Kit supports agent composition and workflow patterns. Its workflow agents can structure sequential, parallel or iterative execution.
Example: Run two specialist agents in parallel, then pass their outputs to a summarizer with an explicit merge contract. [11]

PydanticAI. PydanticAI uses Python types and validation around agents, tools and outputs. Typed contracts improve integration but do not guarantee semantic correctness.
Example: Require an agent to return a validated SupportAnswer object before the application accepts it. [12]

Need	Design feature to inspect
Resume after a restart	Durable state and checkpoints
Review a side effect	Interrupts and exact-payload approval
Change providers	Supported model interfaces and adapters
Diagnose a run	Trace coverage and export options

[9-12] Official framework documentation | Links in Sources & further reading.

Evaluation: judge answers and evidence

Evaluation asks whether the system behaves well on defined tasks. Write criteria and examples before optimizing the prompts or selecting a model.

LLM accuracy. Accuracy measures correctness against a reference or task rubric. Exact-match scoring fits some tasks; open-ended answers often need claim-level or rubric-based checks.
Example: Check whether a policy answer states the correct eligibility rule and exception, not whether it copies the reference wording.

Relevance. Relevance asks whether an answer addresses the user's actual question with appropriate detail. An answer can be factual but irrelevant.
Example: A password-reset answer should give reset steps, not a general essay on cybersecurity.

Hallucination. A hallucination is unsupported or fabricated content presented as fact. Detect it with source checks, executable validation and carefully designed review rubrics.
Example: Flag an answer that invents a reimbursement deadline absent from the policy and other accepted references.

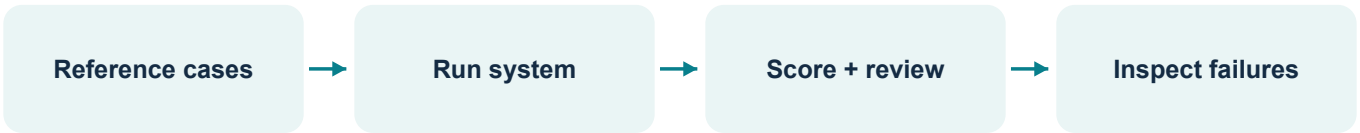
Toxicity. Evaluate abusive, discriminatory or otherwise harmful content in context, including quoted material and dialect variation. Automated classifiers require calibration and human review.
Example: Check that a support assistant does not produce abusive replies even when the incoming ticket contains insults.

LLM-as-judge. A model scores answers using a rubric. Judge scores can be biased by verbosity, position or shared model weaknesses, so calibrate against human labels.
Example: Blindly compare two answers in randomized order and measure agreement between the judge and trained reviewers.

RAG retrieval evaluation. Evaluate candidate retrieval separately from generation, using annotated relevant passages and metrics at a chosen cutoff k.
Example: If the required exception clause is never retrieved, fix retrieval before attempting to improve the answer prompt.

Groundedness / faithfulness. These often-overlapping metrics test whether claims are supported by supplied context and preserve its meaning. Define the exact rubric; grounded claims can still rely on an outdated source.
Example: An answer must retain 'only after approval' when paraphrasing a policy, rather than converting a conditional benefit into a guarantee.

CONCEPT FLOW



In practice Separate truth, relevance and support. A true statement can be unsupported by the supplied context; a faithfully quoted source can itself be wrong or out of date.

Precision, recall and agent success

Choose denominators carefully. A metric name without a definition, sample size and evaluation set is hard to interpret.

Recall. Retrieval recall@k is the number of relevant items retrieved in the first k results divided by all labeled relevant items. It measures how much needed evidence was found.
Example: Four passages are relevant; the top five results include three. Recall@5 = 3/4 = 75%.

Precision. Retrieval precision@k is the number of relevant items in the first k results divided by k. It measures the fraction of returned items that are useful.
Example: The same top five results contain three relevant passages. Precision@5 = 3/5 = 60%.

Task success. Agent task success measures whether the specified task conditions were satisfied. Define partial success and failure before running the evaluation.
Example: Success requires a correctly categorized ticket with valid evidence and no unauthorized actions.

Tool selection. Score whether the chosen tool was appropriate and permitted. A completed task can still use an unnecessarily risky or expensive tool.
Example: Answering a ticket-status question should call a read tool, not modify the ticket merely to discover its status.

Tool arguments. Check required fields, valid values, resource identity and authorization. Schema validity alone does not prove an argument refers to the right target.
Example: ticket_id is an integer, but the evaluator also checks that it is the ticket the user asked about.

Trajectory. Evaluate the observable sequence of decisions, tool calls and outcomes for efficiency, safety and recovery. Do not require disclosure of hidden chain-of-thought.
Example: Count repeated failed searches and check whether the agent stops when its evidence budget is exhausted.

Goal completion. Goal completion measures the end-to-end business outcome, including external state and user requirements. It may coincide with task success or be a broader metric.
Example: Verify that the intended ticket exists and the user received its ID, rather than accepting the agent's statement that it succeeded.



Worked retrieval example: 3 relevant results returned, 5 total returned, 4 relevant in corpus.

In practice For classification: precision = TP/(TP+FP), recall = TP/(TP+FN). A zero denominator needs an explicitly documented scoring convention.

Build an evaluation pipeline

Evaluation is a repeatable process, not a single score at launch. Keep representative cases, failure cases and adversarial cases under version control.

Golden dataset. A golden dataset contains reviewed inputs, expected outcomes, source references and rubrics. Protect a held-out portion from prompt tuning and label changes.
Example: Maintain questions covering routine policies, exceptions, missing evidence, unauthorized access and malicious document text.

Offline evals. Offline evaluations run before deployment on controlled datasets. Pin relevant versions so changes can be compared fairly.
Example: Compare a new chunking strategy with the baseline using identical questions, documents and answer-scoring rules.

Online evals. Online evaluations measure behavior under real usage, with privacy controls and careful interpretation of feedback bias.
Example: Sample production answers for review and correlate complaints with specific model, prompt and index versions.

A/B testing. Randomly assign eligible traffic to alternatives and define outcome, sample size and guardrail metrics in advance. Avoid drawing conclusions from tiny samples or repeated early peeking.
Example: Compare two retrieval strategies on successful resolutions while also monitoring latency and unsupported-claim rate.

Regression testing. Regression tests protect previously working behavior. Include historical bugs and security failures so a change cannot quietly reintroduce them.
Example: A test ensures that a revoked document is absent from retrieval and citations after an index update.

Human evaluation. Human reviewers use a shared rubric and calibrated examples. Double-label a subset and resolve disagreements to improve the rubric and estimate label quality.
Example: Two reviewers independently score whether an answer states a policy exception correctly, then adjudicate differing labels.

Illustrative release gate	Why it matters
No failures on critical access-control tests	Prevent known high-impact regressions
Task-success rate meets agreed baseline	Protect utility
Unsupported-claim rate within target	Protect answer reliability
Latency and cost within agreed budgets	Protect operating constraints

In practice Practice: create 30 reviewed cases, reserve 10 as held out, compare two prompts on the other 20, then run the held-out set once. This is a learning exercise, not a production sample-size recommendation.

Observability: traces, metrics and logs

Observability helps explain what happened in a running system. Evaluation asks how good the result was; the two disciplines complement each other.

Agent traces. A trace links the operations in one run, making sequence, timing and parent-child relationships visible. Include run status and version metadata.
Example: Follow one user question through routing, retrieval, model generation and ticket creation.

LLM calls / tool calls. Record model and tool operations as separate spans with duration, status and selected metadata. Redact sensitive content and control retention.
Example: A trace shows that a slow answer spent 200 ms retrieving evidence and 4 seconds waiting for generation.

Retrieval / handoffs. Trace query processing, selected document IDs, index version and agent transitions. Record why a route was chosen in a concise, observable decision summary.
Example: A poor answer can be traced to a handoff that omitted the user's regional constraint.

Latency. Measure elapsed time, including queueing and dependencies. Track percentiles such as p50 and p95, plus time to first token and total completion time.
Example: A low median with a high p95 reveals that a minority of users experience long waits.

Tokens and cost. Track input/output usage and total estimated or billed costs by task, model and tenant. Include retries, tools and hosting to avoid understating cost.
Example: A retry-heavy document workflow costs twice as much as suggested by counting only its final model call.

Success rate / failure rate. Define categories and denominators explicitly. Technical success, evaluated task success and user satisfaction are different signals.
Example: An HTTP 200 response containing an unsupported answer is a technical success but an answer-quality failure.

Errors, requests and tool logs. Logs record discrete events such as a request accepted, timeout raised or tool result received. Correlate them with trace and request IDs.
Example: A tool log captures a permission-denied error code without copying the protected document text.

Audit logs. Audit logs record who did what, to which resource, when and under which authority. Protect them from unauthorized changes and separate them from verbose debugging data.
Example: Record the approved ticket payload, approver identity and resulting ticket ID for later accountability.

In practice Privacy by design: log IDs and safe metadata by default. Capture full prompts or tool outputs only where justified, access-controlled and subject to retention rules.

OpenTelemetry and observability platforms

Use consistent instrumentation across models, tools and services so incidents can be followed across system boundaries.

OpenTelemetry traces and spans. OpenTelemetry provides vendor-neutral instrumentation and telemetry collection. A trace is a complete request path; each span is a timed operation with attributes and relationships.
Example: Create a parent ask-request span with child retrieval, generation and tool-execution spans. [13]

OpenTelemetry metrics. Metrics summarize measurements such as counts, histograms and gauges. Choose low-cardinality labels; request IDs usually belong in traces or logs, not metric labels.
Example: Record a latency histogram by service and model family rather than creating a separate metric series for every user.

Semantic conventions. Semantic conventions standardize attribute names and meanings across instrumentation. GenAI conventions evolve; pin and check the version used by your exporters and backend.
Example: Use consistent attributes for model identity, operation type and token usage so dashboards compare like with like.

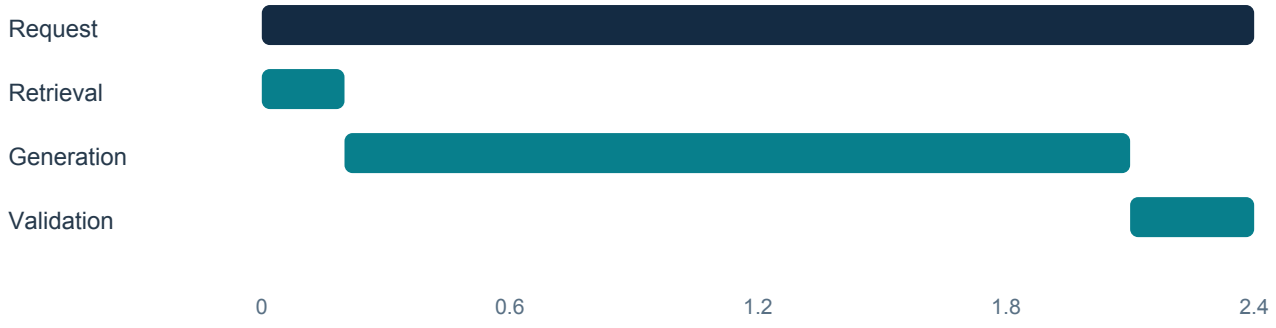
LangSmith. LangSmith provides application tracing and evaluation workflows, including analysis of model and tool execution. Check integration and retention needs against your deployment.
Example: Inspect a failed retrieval-and-answer run, then turn it into an evaluation case. [14]

Langfuse. Langfuse is an AI engineering platform with tracing, evaluation and prompt-management capabilities. Hosted and self-managed choices have different operating implications.
Example: Connect prompt versions to latency, cost and user feedback to investigate a regression. [15]

Arize Phoenix. Phoenix provides open-source tooling for AI observability and evaluation, including tracing and retrieval-oriented inspection.
Example: Inspect which retrieved passages were used and label problematic outputs for follow-up evaluation. [16]

Cloud observability. Cloud monitoring platforms aggregate service logs, metrics, traces and alerts near deployed infrastructure. AI-specific quality checks still require explicit instrumentation.
Example: Correlate a rise in model timeouts with container restarts and outbound network errors in the same time window.

ILLUSTRATIVE REQUEST TRACE / SECONDS



In practice The trace below is illustrative: a 2.4-second request contains sequential 0.2-second retrieval, 1.9-second generation and 0.3-second validation spans.

[13-16] Official observability documentation | Links in Sources & further reading.

Threats: instructions can arrive inside data

Security starts by separating trusted control instructions from untrusted content. Retrieved pages, attachments and tool results can contain malicious instructions. [17]

Prompt injection. Prompt injection attempts to make a model follow attacker-controlled instructions that conflict with the intended task or policy. Treat it as a system problem, not only a wording problem.
Example: A user asks the assistant to ignore application rules and reveal protected records; authorization checks must still deny access.

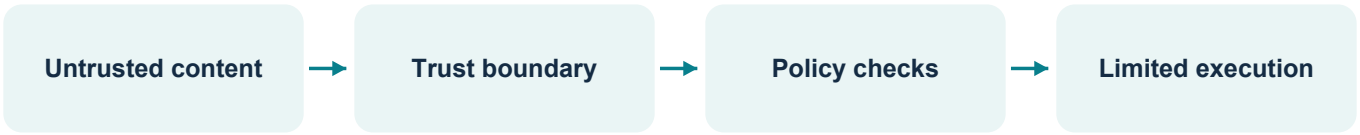
Indirect injection. Indirect injection arrives through content the system reads, such as a web page, document or tool response. The user may have no knowledge of the malicious content.
Example: A retrieved policy PDF includes a line asking the agent to upload private files; that line remains untrusted document data.

Jailbreaking. Jailbreaking attempts to bypass a model's behavioral restrictions. It overlaps with some injection attacks, but often targets the model's response policy directly.
Example: A role-play prompt tries to elicit prohibited behavior; evaluate the resulting system behavior without relying solely on a blocklist.

Data leakage. Sensitive data can escape through generated answers, tool calls, caches, logs or training pipelines. Track trust boundaries and minimize data shared across each one.
Example: An answer cache accidentally serves one organization's internal policy to another; partition cache eligibility by tenant and permissions.

PII detection. Personally identifiable information detection locates data that identifies people, using patterns, classifiers or contextual rules. Detection has false positives and false negatives.
Example: Redact email addresses and account identifiers from debugging samples, then review edge cases that automated detection may miss.

CONCEPT FLOW



In practice No prompt alone makes an agent secure. Reduce what tools can do, validate actions outside the model, constrain data movement and test the complete application.

[17] OWASP: Prompt Injection | Links in Sources & further reading.

Guardrails, permissions and isolation

Guardrails reduce risk around inputs and outputs. Authorization and isolation enforce boundaries even when the model is mistaken.

Input guardrails. Inspect incoming data for size, format, task suitability and known abuse patterns. Preserve a clear boundary between the user's request and quoted or retrieved data.
Example: Reject an oversized upload and quarantine an unsupported file type before parsing it.

Output guardrails. Validate generated structure and content before release or execution. Check sensitive data, source support and task-specific constraints; handle blocked results explicitly.
Example: Reject a tool payload with an unknown recipient and flag an answer whose citations do not support its main claim.

Tool permissions. Define exactly which tools a caller or agent can use and on which resources. Enforce these permissions in the execution layer on every call.
Example: The support assistant may create a normal ticket but cannot delete a user account.

Least privilege. Grant only the minimal capabilities and data needed for the current task, preferably with short lifetimes and narrow scopes.
Example: Use a read-only database role for question answering and a separate restricted service for ticket creation.

Secrets management. Store credentials in a managed secret store or equivalent secure mechanism; inject them where needed and rotate them. Keep them out of prompts, repositories and logs.
Example: The backend authenticates to the model provider without ever sending its API key as part of the model's context.

Agent sandboxing. Isolate code, files, network destinations and compute resources. Containers alone may not provide sufficient isolation for arbitrary untrusted execution.
Example: Run document-processing code with a read-only input mount, a dedicated output directory, no production secrets and a time limit.

Boundary	Enforced control	Example
Data -> model	Permission-filtered retrieval	Only employee-visible chunks
Model -> tool	Schema + authorization	Approved ticket operation
Runtime -> network	Egress restrictions	Approved service destinations
Output -> user	Policy and evidence checks	No cross-tenant data

In practice Defense in depth means independent layers. If the model ignores an instruction, the tool executor should still refuse unauthorized access.

MCP security and adversarial testing

Integration protocols increase reach. Review the servers, tool definitions, credentials and destinations that become part of your application's trust boundary.

MCP security. Validate server provenance, tool metadata, identity and requested scopes. Review tool changes and untrusted results; do not assume protocol compatibility means safety.
Example: A newly configured server is allowed to expose read-only document search, while arbitrary command execution remains unavailable.

Rate limiting for security. Limit requests, tokens, concurrent tasks and resource-heavy actions to reduce abuse and denial-of-service risk. Apply limits across identities and workloads where appropriate.
Example: Cap expensive OCR jobs per organization and stop an agent run after its tool-call or spending budget is reached.

AI red teaming. Red teaming systematically tests adversarial scenarios in an authorized environment. Vary language, document content, tool errors and multi-step interactions, then convert failures into regression cases.
Example: Place a harmless instruction-like string in a test document and verify that the assistant treats it as content rather than following it.

People, process and governance. Assign owners for approvals, incident response, access reviews and change management. Keep evidence of decisions and make emergency shutdown practical.
Example: A documented incident process identifies who can disable a compromised tool integration and how affected users are notified.

Test case	Expected behavior	Evidence to record
Document contains hostile instructions	Answer task; do not execute them	Trace and denied tool request
User requests another tenant's data	Deny or omit protected data	Authorization decision
Tool returns unexpected schema	Reject / handle failure	Validation error
Repeated expensive requests	Throttle within policy	Quota event
Approval payload changes	Require renewed approval	Payload hash / version

In practice Practice: use synthetic secrets and test accounts. A successful test means the boundary held, not that every possible attack has been eliminated.

GenAI Ops / AgentOps: reproducible releases

MLOps manages model lifecycles; LLMOps adds prompts, retrieval and model APIs; AgentOps adds tools, state and action trajectories. Their boundaries overlap.

GitHub Actions. GitHub Actions automates repository-triggered jobs such as linting, tests, builds and deployments. Scope workflow permissions and protect deployment credentials.
Example: A pull request runs unit tests and a small evaluation suite, then publishes its results for review.

CI/CD. Continuous integration checks changes together; continuous delivery or deployment moves approved artifacts through release stages. AI gates should cover quality and security as well as code.
Example: A prompt update reaches staging only after regression evaluations and permission-boundary tests pass.

Docker in production. Build immutable images with pinned dependencies, minimal runtime contents and health checks. Record the digest used in each release.
Example: Deploy exactly the image evaluated in staging rather than rebuilding it with potentially changed dependencies.

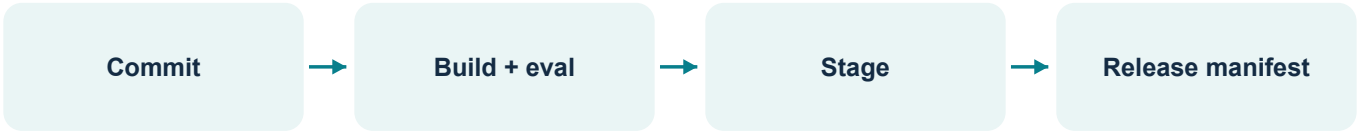
Kubernetes. Kubernetes schedules and manages containerized workloads, service discovery and rollout behavior. It is useful at scale but adds operational complexity.
Example: Run separate API and ingestion-worker deployments with resource limits and independent scaling policies.

Model versioning. Track model identifiers, weight hashes or provider deployment versions. Hosted aliases may change; record what can actually be pinned and when a run occurred.
Example: A release manifest names the exact self-hosted weight checksum or the available provider model/deployment identifier.

Prompt versioning. Version the full instruction template, examples and generation settings, not just a prompt name. Connect runs and evaluation results to that version.
Example: Prompt v12 adds an abstention rule; compare its errors and success rate with v11 before promotion.

Dataset versioning. Version training, evaluation and retrieval datasets, including transformations, labels and permission metadata. Use stable IDs and reproducible snapshots.
Example: Evaluation set v4 corrects two labels while retaining a record of the prior scores and why labels changed.

CONCEPT FLOW



In practice A useful release manifest includes code, image, model, prompt, tool schema, dataset and index versions. These components can interact, so rolling back one may not restore compatibility.

Operate, recover and scale

Operational readiness means detecting deterioration, limiting its impact and recovering without guessing which component changed.

Experiment tracking. Record hypotheses, configurations, artifacts and results so experiments can be compared and repeated. Include failed experiments, not only winning runs.
Example: Compare chunk sizes with the same query set, recording recall, groundedness, latency and cost for each run.

Eval pipelines. Automated evaluation pipelines run appropriate test suites on changes and scheduled checks. Fail critical gates and route ambiguous quality changes to review.
Example: A new model deployment must pass access-control tests and meet agreed answer-quality thresholds before release.

Rollbacks. A rollback restores a known-good compatible release. Plan for schema, state and index changes; some migrations require forward fixes or explicit compatibility support.
Example: Switch the application back to prompt v11 and index v6 together because prompt v11 expects that index's citation format.

Canary deployment. A canary sends a small controlled share of traffic to a candidate release, with monitoring and stop criteria. Keep assignment stable where sessions need consistency.
Example: Expose 5% of eligible sessions to a new routing policy and stop if its error or cost guardrail is breached.

Autoscaling. Autoscaling adjusts capacity to demand using signals such as queue depth, utilization or request rate. Model-provider quotas and cold starts can remain bottlenecks.
Example: Increase ingestion workers when queue age grows, while keeping model API concurrency below the approved quota.

Cost optimization in operations. Use capacity planning, right-sizing, batching, cache policy and retention controls. Include idle hardware, storage, network egress and retries in the operating budget.
Example: Schedule overnight batch embeddings and remove obsolete indexes instead of paying continuously for unused replicas and storage.



Illustrative rollout allocation, not a universal recommended percentage.

In practice Rollback drill: deliberately fail a staging release, restore the previous manifest and confirm that pending jobs resume without duplicate actions.

Cloud AI engineering: AWS

Choose services by workload, existing infrastructure, identity, region and operational needs. The screenshots name product families; their marketing comparisons are not objective rankings.

Amazon Bedrock. Bedrock provides managed access to foundation models and related generative AI capabilities. Model availability, interfaces and controls vary by region and model.
Example: Use an approved managed model for the knowledge assistant while retaining application-side retrieval and authorization. [18]

SageMaker. The screenshot's SageMaker label covers an ML platform family; Amazon SageMaker AI supports building, training and deploying models. It is useful when managing custom ML workflows.
Example: Train a ticket classifier on company labels and deploy a versioned inference endpoint. [19]

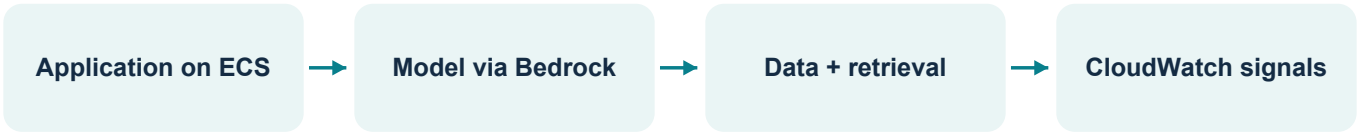
ECS. Amazon Elastic Container Service orchestrates container workloads. It offers an AWS-native path for running services without adopting Kubernetes APIs.
Example: Run the FastAPI service and background ingestion workers as separate container services. [20]

EKS. Amazon Elastic Kubernetes Service provides managed Kubernetes. It suits teams needing Kubernetes APIs, ecosystem tools or established cluster operations.
Example: Deploy API, worker and model-serving workloads into an existing Kubernetes platform. [21]

Lambda. AWS Lambda runs functions in response to requests or events with managed infrastructure. Execution limits, cold starts and packaging affect workload fit.
Example: A new file event starts an ingestion task; long processing is handed to a suitable job or worker service. [22]

CloudWatch. CloudWatch collects and analyzes operational signals such as metrics and logs, supporting monitoring and alarms across AWS resources.
Example: Alert when ingestion queue age rises or the assistant's error rate exceeds its defined threshold. [23]

CONCEPT FLOW



In practice Illustrative composition, not a required stack: replace ECS with EKS or a function-based design when workload and team constraints justify it. Identity, data storage and networking must also be designed.

[18-23] AWS official service overviews | Links in Sources & further reading.

Cloud AI engineering: Google Cloud

Separate the model family from the platform that hosts, deploys or manages workloads. They are related choices, not interchangeable products.

Vertex AI. Vertex AI is the platform name shown in the screenshot for managed ML and generative AI workflows. The referenced overview currently redirects to Gemini Enterprise Agent Platform documentation; check current naming and migration guidance.
Example: Use managed training and a model registry to track and deploy a ticket classifier. [24]

Gemini. Gemini is Google's model family, used for language and supported multimodal tasks through available APIs and platforms. Choose a specific model by evaluated capabilities.
Example: Analyze a support screenshot together with the user's question and retrieve relevant troubleshooting evidence.

Cloud Run. Cloud Run is a managed platform for running applications and containers, including request-serving services and jobs. It reduces cluster-management work but still requires application configuration.
Example: Deploy the assistant's HTTP API as a service and run document processing as a separate job. [25]

GKE. Google Kubernetes Engine manages Kubernetes infrastructure. Use it where cluster-level control and Kubernetes workload patterns fit the team's needs.
Example: Host GPU inference and supporting workers on an established Kubernetes platform. [26]

BigQuery. BigQuery is a managed analytics data platform suited to large analytical queries. It is not a direct replacement for an application's transactional database.
Example: Analyze anonymized usage events to find the highest-cost task types and measure changes in resolution rates. [27]

Workload	Screenshot service	Example
Foundation-model inference	Gemini / managed AI platform	Generate grounded answers
Managed ML lifecycle	Vertex AI label	Train and register classifier
Application runtime	Cloud Run	Serve HTTP API
Kubernetes runtime	GKE	Run cluster workloads
Analytics	BigQuery	Analyze usage and quality

In practice Terminology checked on 12 October 2026. Keep the screenshot labels for learning continuity, but verify current console names, APIs, supported regions and deployment options before implementation.

[24-27] Google Cloud official documentation | Links in Sources & further reading.

Cloud AI engineering: Azure

Azure services can combine managed models, application compute and enterprise identity. Service choice still depends on measured requirements.

Azure AI. Azure AI is the broad label used in the screenshot for AI services and development capabilities. Current platform documentation describes Microsoft Foundry; the naming and product organization have evolved.
Example: Build and evaluate a company assistant using the platform's model and application tooling. [28]

Azure OpenAI. Azure OpenAI provides access to supported OpenAI models within Azure's deployment and identity environment. Its model catalog and API behavior must be checked for the selected deployment.
Example: Use an approved model deployment for generation while applying the organization's Azure access and networking controls.

AKS. Azure Kubernetes Service is managed Kubernetes for container workloads. Cluster policies, node pools and scaling settings remain important design choices.
Example: Run a retrieval API and separate background processors in existing Kubernetes infrastructure. [29]

Functions. Azure Functions runs event-driven application code on managed hosting options. Match the hosting plan and execution limits to the workload.
Example: Process a document-upload event and enqueue a longer extraction job. [30]

Azure Monitor. Azure Monitor collects and analyzes application and infrastructure telemetry, with associated logs, traces and alerts.
Example: Correlate failed model calls with application exceptions and container resource pressure. [31]

Decision dimension	What to compare across clouds
Data and identity	Existing access controls, residency and network paths
Models	Task quality, modality support and regional availability
Runtime	Request duration, scaling, GPUs and operational effort
Reliability	Quotas, failure recovery, dependencies and support
Economics	Model usage, idle compute, storage and network egress

In practice Cloud services are not perfect one-to-one equivalents. Compare a complete deployment against the same requirements rather than comparing product names alone.

[28-31] Microsoft official documentation | Links in Sources & further reading.

System design: start with the outcome

Architecture design maps requirements to components, contracts and failure behavior. Begin with users, data boundaries, expected traffic and measurable quality goals.

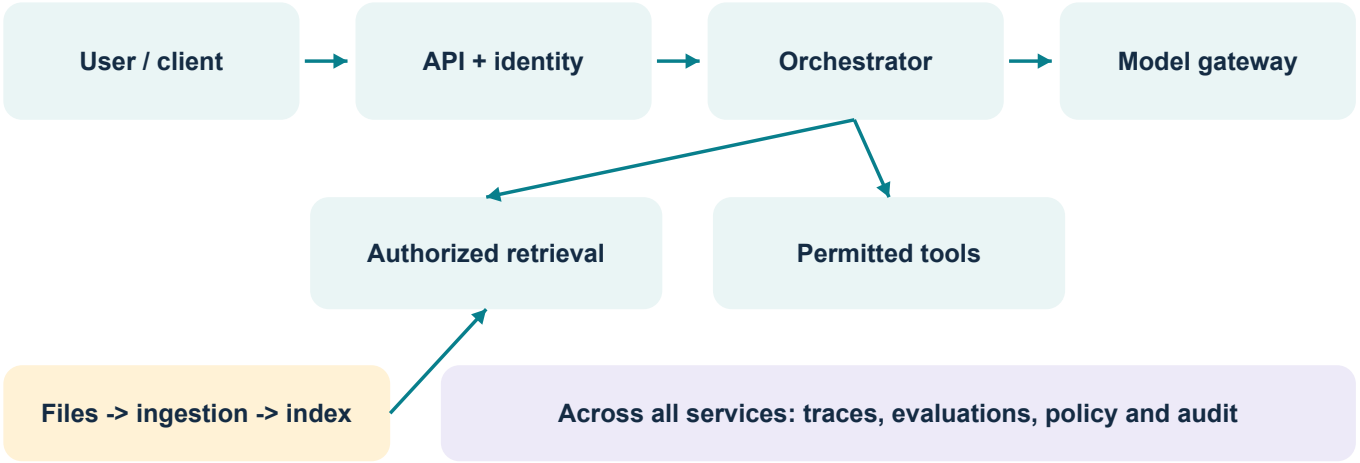
Architecture design. Separate request handling, retrieval, model access, tools and background work behind explicit interfaces. Draw data flows and trust boundaries, including failure and recovery paths.
Example: The assistant has an API, a permission-aware retriever, a model gateway, a ticket tool and an asynchronous ingestion pipeline.

Model selection in system design. Select models within the complete system's quality, latency, privacy and cost budgets. Measure end-to-end performance rather than relying on isolated model benchmarks.
Example: A fast model plus strong retrieval may meet the answer-quality target more cheaply than a larger model given an oversized context.

RAG vs fine-tuning. RAG supplies external evidence during inference; fine-tuning changes model behavior through training. Use RAG for changing facts and citations; tuning may improve stable task behavior. They can be combined.
Example: Retrieve the current policy while using a tuned model to consistently classify requests and format responses.

Agent vs workflow. A workflow follows predefined control logic; an agent makes some runtime choices through a model. Use the simpler design that handles the task's uncertainty.
Example: A fixed upload-parse-index pipeline is a workflow; investigating an ambiguous incident may need bounded agent-selected searches.

Database selection. Choose stores by access pattern, consistency, query needs and operational cost. Relational, key-value, object, vector and graph storage solve different problems.
Example: Use PostgreSQL for tickets, object storage for PDFs, a vector index for passages and Redis for eligible cache entries.



In practice Reference architecture: retrieval and tool execution both enforce authorization. Observability and evaluation collect signals across the application, rather than being a final processing step.

Queues, events and caching

Decouple work when it improves responsiveness or recovery. Every asynchronous boundary needs ownership, delivery semantics and an observable status.

Event-driven architecture. Producers publish facts about changes; consumers react without requiring a single synchronous request chain. Schema evolution and event ordering need explicit handling.
Example: A document-uploaded event triggers extraction, and an indexing-complete event marks the document searchable.

Queues. Queues buffer work and absorb bursts. Use acknowledgments, visibility timeouts, retries and dead-letter handling; common delivery modes can deliver a job more than once.
Example: A worker uses document_id plus version as an idempotency key so a retried ingestion job does not create duplicate chunks.

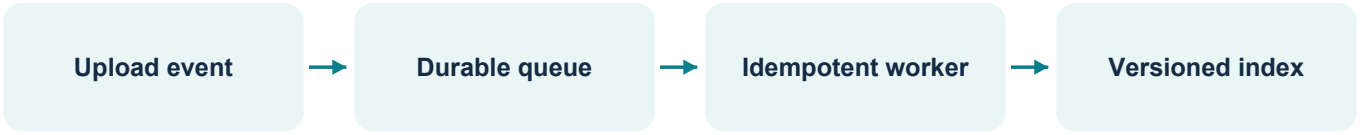
Caching. Caching reuses expensive results using a key and invalidation policy. Different layers include HTTP, exact-response, semantic-answer, embedding and provider prompt caches.
Example: Reuse an embedding for the same text and embedding-model version; invalidate answers when their policy version changes.

Reliability. Reliability means consistently delivering correct service under stated conditions. Set service-level indicators and objectives for availability, correctness and timeliness.
Example: Track the share of eligible requests that return a supported answer within the agreed deadline.

Fault tolerance. Fault tolerance keeps useful behavior available despite component failures. Apply timeouts, bounded retries, circuit breakers, redundancy and graceful degradation where appropriate.
Example: If the ticket service fails, preserve the approved draft and report that submission is pending rather than claiming success.

Scalability. Scalability is the ability to handle more load without unacceptable loss of performance or cost efficiency. Find bottlenecks across compute, databases, queues and external quotas.
Example: Adding API replicas does not improve throughput when all requests are blocked by the same model-provider token limit.

CONCEPT FLOW



In practice Delivery lesson: an at-least-once queue plus an idempotent consumer is often more practical than claiming exactly-once behavior across all external systems.

Security, observability and latency budgets

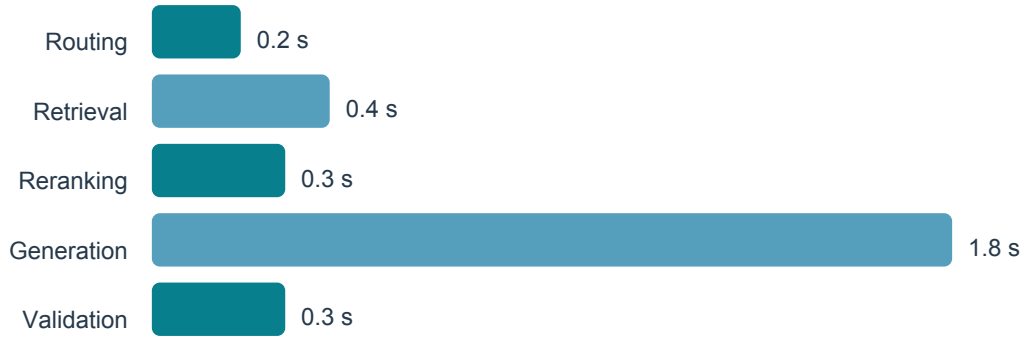
Cross-cutting requirements must be part of the architecture from the beginning. A diagram is incomplete if it omits authorization, recovery and measurement.

Security. Threat-model identities, data stores, model context and side-effecting tools. Apply tenant isolation, least privilege, encryption and auditability at the actual enforcement points.
Example: Document permissions are checked in retrieval and again when opening a citation, preventing a link from bypassing access control.

Observability in system design. Propagate request and trace context across services and queues. Record versions and safe metadata so an incident can be localized and reproduced.
Example: A quality regression can be filtered by prompt version and then linked to a changed retrieval index.

Latency optimization. Measure where time is spent, then reduce unnecessary work. Parallelize independent I/O, shorten context/output, cache safely and stream where appropriate.
Example: Retrieve two independent sources in parallel; do not parallelize a restricted read before its authorization decision.

Latency budgets. Allocate an end-to-end target across queueing, retrieval, reranking, generation and validation. Tail latency can compound across dependent services.
Example: For an illustrative 3-second target, budget 0.2 seconds for routing, 0.4 for retrieval, 0.3 for reranking, 1.8 for generation and 0.3 for validation.



Illustrative sequential latency budget: 3.0 seconds total, excluding unbudgeted queueing.

In practice Streaming can improve time to first visible output without reducing total completion time. Report both metrics; do not call an unfinished answer a completed task.

Cost engineering: measure useful outcomes

Cost engineering connects technical choices to a full operating model. Cheap requests are not valuable if they repeatedly fail the user's task.

Cost engineering. Estimate model, retrieval, storage, compute, network and operational costs, then measure actual usage. Separate fixed capacity from variable usage and budget for retries and growth.
Example: A self-hosted model may lower marginal token cost but add GPU idle time, capacity planning and maintenance.

Per-request model cost. Multiply each billable unit by its rate using compatible units. Include additional reasoning, cache or modality charges where the provider applies them.
Example: At fictional rates of $2 per million input tokens and $8 per million output tokens, 3,000 input plus 500 output tokens costs $0.010.

Cost per successful task. Divide the total cost of all attempts, including failures, by completed successful tasks. This captures the impact of retries and quality differences.
Example: If 1,000 attempts cost $20 and 800 meet the success criteria, cost per success is $20/800 = $0.025.

Capacity and concurrency. Estimate throughput, request duration and burstiness together. Little's Law relates average in-flight work, arrival rate and time in a stable system; tails need extra planning.
Example: At 10 requests per second and 3 seconds average duration, average in-flight work is about 30 requests.

Trade-off decisions. Compare changes on quality, latency, cost and risk using the same workload. State the constraints and reject improvements that break a hard requirement.
Example: Removing reranking saves 300 ms but is unacceptable if it pushes evidence accuracy below the agreed quality threshold.

Fictional calculation	Amount
Input: 3,000 / 1,000,000 x $2	$0.006
Output: 500 / 1,000,000 x $8	$0.004
Model total per attempt	$0.010
10,000 identical attempts	$100.00
Plus tools, hosting, retrieval and retries	Measure separately

In practice All prices and traffic numbers on this page are invented for arithmetic practice. They are not vendor quotes or production forecasts.

Capstone: build a trustworthy support assistant

Use one coherent project to connect the entire roadmap. The assistant answers policy questions with citations and creates support tickets when authorized.

Milestone 1: a typed service. Build a small API with validated requests, structured responses and authenticated user identity. Use a relational store for tickets and a safe configuration system.
Example: Accept a question and return a mock answer; test invalid input and forbidden resource access before adding a model.

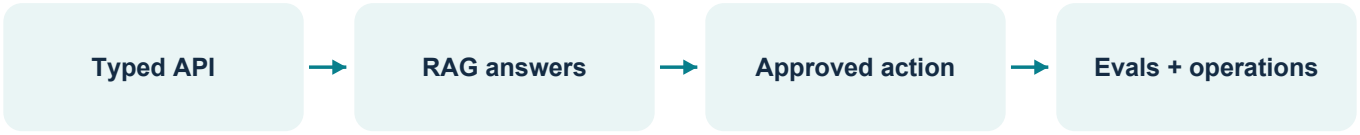
Milestone 2: evidence-based answers. Implement document ingestion, permission-aware retrieval, context building and citation checks. Start with a simple retrieval baseline.
Example: Answer questions from ten sample documents, including one stale policy and one document inaccessible to the test user.

Milestone 3: controlled actions. Add a narrow ticket tool with argument validation, an exact-payload approval step and an idempotency key. Persist action status.
Example: After approval, create one ticket and return its real ID; replay the same request and confirm no duplicate appears.

Milestone 4: measurable quality. Build reviewed evaluation cases for answer correctness, retrieval, abstention, permissions and tool behavior. Record baseline results.
Example: Include a missing-answer question, a malicious instruction embedded in a document and a simulated tool timeout.

Milestone 5: production behavior. Containerize the system, trace requests, define quotas, version release components and practice rollback and recovery.
Example: Restart a worker mid-task and confirm it resumes safely, keeping the user informed of any incomplete action.

CONCEPT FLOW



In practice Definition of done: a correct cited answer or honest abstention; authorized actions only; no duplicates on retry; reproducible versions; observable failures; measured latency and cost.

Practice and self-check

Work through these exercises in dependency order. The goal is to explain a design choice and demonstrate its behavior, not merely recognize a product name.

Foundations check. Explain why type hints, runtime validation and database constraints each catch different errors. Then demonstrate one example of each.
Example: A type checker finds a wrong argument type; validation rejects an empty question; a foreign key rejects a nonexistent owner.

Retrieval check. For a query with six relevant passages, retrieve ten results containing four relevant passages. Calculate precision and recall, then suggest a targeted improvement.
Example: Precision@10 = 4/10 = 40%; recall@10 = 4/6, about 66.7%. Improve candidate recall before only adjusting answer wording.

Agent check. Explain what happens if a worker crashes immediately after creating a ticket but before saving its checkpoint.
Example: On resume, reconcile the idempotency key or external action record; do not blindly create the ticket again.

Security check. Explain why filtering a malicious sentence from a document is not sufficient protection against unauthorized tool use.
Example: The executor must still enforce permissions, validate arguments and constrain destinations even if input detection misses the attack.

Operations check. A new model improves offline accuracy but increases p95 latency and doubles cost per successful task. Decide whether it should ship.
Example: Compare the result with agreed quality, latency and budget constraints; test targeted routing before adopting it for every request.

Design check. Choose between RAG, fine-tuning and a deterministic workflow for a frequently changing policy assistant with an approval-based ticket process.
Example: Retrieve policies with RAG, use a workflow for approval and submission, and consider tuning only for measured stable behavior gaps.

Learning stage	Demonstrable outcome
1. Foundations	Validated API with tested data handling
2. Models and context	Measured prompt/model baseline
3. Retrieval and tools	Cited answers and controlled actions
4. Evaluation and safety	Reviewed cases and failure tests
5. Deployment and design	Recoverable, monitored release

In practice Coverage: chapters 1-13 follow screenshots 1-13 in order. Repeated topics are explained in their different contexts; chapter 14 adds practice and a unifying project.

Sources & further reading / 1

Primary references for the concepts and product descriptions. Links were checked on 12 October 2026. Product documentation may redirect as naming evolves.

[1] FastAPI documentation
Python API framework, validation and typed interfaces.

[2] PyTorch: Learn the Basics
Tensors, automatic differentiation and model training.

[3] Vaswani et al. - Attention Is All You Need
Original transformer architecture paper (2017).

[4] Hu et al. - LoRA
Low-rank adaptation of large language models (2021).

[5] Lewis et al. - Retrieval-Augmented Generation
Original RAG research paper (2020).

[6] Yan et al. - Corrective RAG
Retrieval evaluation and corrective retrieval (2024).

[7] Asai et al. - Self-RAG
Trained retrieval and critique with reflection tokens (2023).

[8] Model Context Protocol - Architecture
Hosts, clients, servers and protocol capabilities.

[9] LangGraph overview
Stateful graph-based orchestration and durable execution.

[10] OpenAI Agents SDK
Agents, tools, handoffs, guardrails and tracing.

[11] Google ADK - Workflow agents
Workflow patterns and composition.

Sources & further reading / 2

Primary references for the concepts and product descriptions. Links were checked on 12 October 2026. Product documentation may redirect as naming evolves.

[12] PydanticAI - Agents
Typed agent abstractions, tools and outputs.

[13] OpenTelemetry - Observability primer
Signals, instrumentation and observability concepts.

[14] LangSmith - Observability
Tracing and AI application inspection.

[15] Langfuse documentation
Tracing, evaluation and prompt-management capabilities.

[16] Arize Phoenix documentation
AI observability, tracing and evaluation.

[17] OWASP - Prompt Injection
Direct and indirect prompt-injection risks and mitigations.

[18] Amazon Bedrock overview
Managed foundation-model access.

[19] Amazon SageMaker AI overview
Building, training and deploying ML models.

[20] Amazon ECS overview
Managed container orchestration.

[21] Amazon EKS overview
Managed Kubernetes on AWS.

[22] AWS Lambda overview
Event-driven functions and execution model.

Sources & further reading / 3

Primary references for the concepts and product descriptions. Links were checked on 12 October 2026. Product documentation may redirect as naming evolves.

[23] Amazon CloudWatch overview
Operational monitoring, metrics and logs.

[24] Google Cloud - Managed ML platform overview
Screenshot label: Vertex AI. Linked overview redirects to current platform documentation.

[25] Cloud Run overview
Managed application, service and job runtime.

[26] Google Kubernetes Engine overview
Managed Kubernetes on Google Cloud.

[27] BigQuery overview
Managed analytics data platform.

[28] Microsoft Foundry overview
Current platform overview for Azure AI development and models.

[29] Azure Kubernetes Service overview
Managed Kubernetes on Azure.

[30] Azure Functions overview
Event-driven application execution.

[31] Azure Monitor overview
Application and infrastructure telemetry.

In practice Source basis: the 13 user-provided screenshots define the topic list. Explanations, diagrams, examples and exercises are original educational synthesis. Numerical scenarios are illustrative unless explicitly stated otherwise. The partially obscured MCP card is identified in chapter 6.